QuantBacktest

Financial Strategy Backtesting Platform

Alexandre Tung Victor Haegeman Eloi Dumas
Alexis Launay-Schapman

EFREI Paris, ST2OOS, 2026/2027

Source code:
`https://github.com/Alexarmanto/QuantBacktest-Financial-Strategy-Backtesting-Platform`

1 Introduction

A backtest answers one question: if I had followed this trading rule in the past, what would have happened to my money? We built a backtesting website in C# (.NET 10, ASP.NET Core MVC) that fetches prices from the Alpha Vantage API, keeps them locally, simulates trading strategies on daily and weekly data, measures their performance and generates reports. The user interface is shown in the demonstration video, so this report focuses on design choices and results.

The application covers every requirement of the subject, with 10 strategies and a buy & hold baseline where 6 were asked for. We added features that make a result easier to trust: stop-loss, take-profit, trailing stop and slippage; an optimisation heatmap with an overfitting warning; train/test validation; a strategy comparator; Monte Carlo simulation; a data quality panel; a no-code strategy builder; permanent links; and a Docker image.

Member	**Responsibility**
Alexandre Tung	Alpha Vantage client, SQLite cache, data quality, PDF and CSV reports, Docker
Victor Haegeman	Indicators, strategies, no-code rule engine, Monte Carlo
Eloi Dumas	Backtesting engine, advanced orders, metrics, optimisation heatmap
Alexis Launay-Schapman	Web application, comparator, train/test, permanent links

Table 1: Split of the work.

2 Architecture

With four people working at the same time, our priority was to avoid blocking each other. The solution has one shared project, `Core`, and one project per responsibility (Figure 1). `Core` only contains interfaces, data models and constants. What a signature cannot say, such as the price at which an order is filled, is written in a contract with numbered rules (X1–X15 for execution, M1–M14 for metrics, D1–D8 for data…) that the code and the tests quote. We froze this contract early and never had to change it.

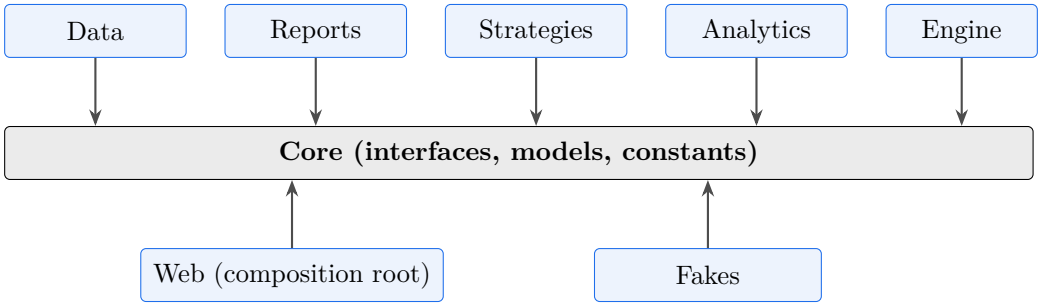


Figure 1: Every project depends on `Core` only. `Web` also references the others, only to plug them in.

A project never references the project of another part: parts communicate through interfaces injected by dependency injection. For a backtest, the web layer calls `IBacktestService`, which gets the strategy from `IStrategyCatalog` and the prices from `IMarketDataService`, then runs `IBacktestEngine`. The resulting `BacktestResult` is displayed and is the input of the Monte Carlo service and of the report generator. A `Fakes` project implements every interface, so the web pages could be built before the real engine existed; one setting per part switches between the fake and the real service.

Some conventions removed whole families of bugs: `decimal` for money and `double` for statistics; fractions in the code and percentages only on screen; invariant culture for parsing; an injected clock; and `null` (shown as "n/a") for an undefined ratio, never `NaN` or 0.

3 Data layer

What the free plan gives. We first tested the endpoints with a free key. The full daily history and the adjusted daily series are premium: we only get the last 100 raw daily bars. The adjusted weekly series, however, is free and goes back to 1999 (1,406 bars), so long tests use weekly data.

Cache and seed files. Prices are stored in SQLite with EF Core. Without a refresh, the cache is returned with no API call, so a backtest is reproducible. If the cache is empty, a committed seed file is imported (AAPL, MSFT, SPY, JPM), or else the API is called. A refresh merges bars by date and never deletes one; if it fails, the cache is unchanged. Seed files also let the application run without a key.

Pitfalls of the API. The project guide gave a good starting point, but we met three traps. Alpha Vantage reports errors as JSON with HTTP 200, so testing the status code would save an error message as data: our client detects and classifies these answers. `double.Parse("123.45")` without a culture misreads the number on a French system, so we always use the invariant culture, and a test runs under `fr-FR`. Finally, the key is in the URL and even in the rate limit message of the API: it is only read from user-secrets or an environment variable, our messages never copy the API text, and HTTP logging is removed. After a real refresh in Docker, the key did not appear anywhere in the logs.

Quota and concurrency. The 25 daily requests are counted per UTC day before sending, so failures count too. Simultaneous calls for a series share one download: 10 calls send one HTTP request.

Data quality. A report computed on cached data flags price jumps above 25 %, zero volumes, five identical closes in a row, and calendar gaps. It finds real events (Figure 2): the 11-day gap of September 2001 and the violent moves of JPM during the 2008 crisis. A flag is an invitation to look, not necessarily an error.

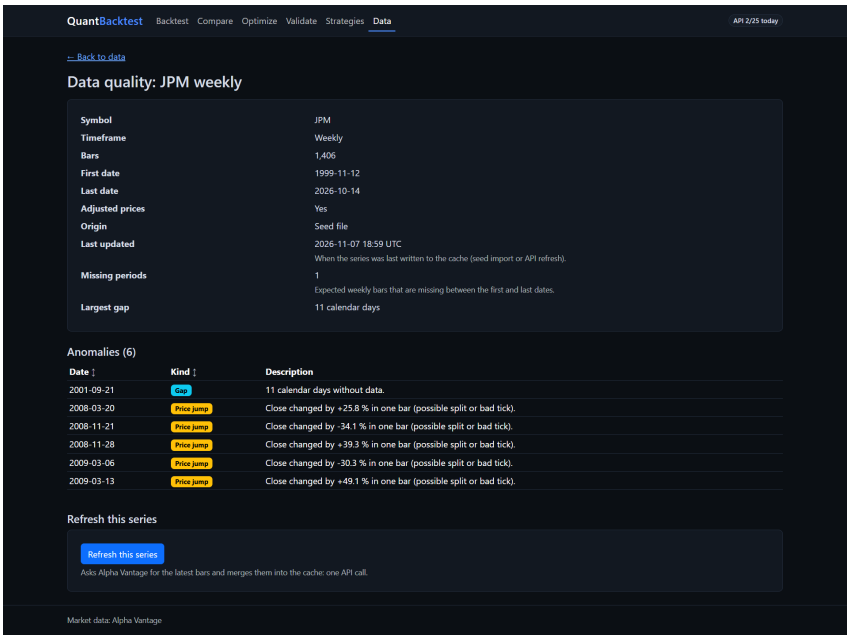


Figure 2: Quality report of JPM weekly: one gap and five price jumps.

4 Strategies

A strategy only returns `Buy`, `Sell` or `Hold` for each bar, and knows nothing about capital, fees or stops. It is deterministic and may only use past bars. All strategies share one implementation of the indicators (SMA, EMA, Bollinger, RSI, ROC, MACD, highest high and lowest low). Classes are discovered by reflection, so adding one does not touch the web project; a commented C# template makes this easy.

Users can also build strategies without code: an entry and an exit rule, each with 1 to 5 conditions combining prices, volume, constants and indicators. A definition is validated, stored as JSON, and becomes a normal strategy.

Table 2 shows the strategies on SPY weekly from 1999 to 2026, with default parameters and a 0.1 % fee. Turn of the month accepts daily data only and is absent.

Strategy	Return	Sharpe	Max drawdown	Trades	Exposure
Moving average crossover	+759.4 %	0.70	31.8 %	9	70.9 %
Donchian breakout	+300.9 %	0.57	27.9 %	22	61.0 %
Buy and hold	+781.5 %	0.55	54.6 %	1	99.9 %
Momentum (ROC)	+258.5 %	0.48	38.5 %	68	68.7 %
Trend-filtered RSI-2	+111.7 %	0.40	23.5 %	42	9.2 %
RSI reversion	+156.9 %	0.37	43.4 %	4	20.6 %
MACD crossover	+94.0 %	0.29	40.9 %	59	53.3 %
Bollinger + RSI	+56.6 %	0.24	37.4 %	6	5.0 %
Bollinger reversion	+67.7 %	0.23	41.4 %	20	14.2 %
Consecutive down days	+30.0 %	0.17	39.8 %	57	7.3 %

Table 2: SPY weekly, 1,406 bars, default parameters, sorted by Sharpe ratio.

Only the moving average crossover comes close to buy & hold, with a much smaller drawdown because it stays out of long declines. Mean reversion strategies are rarely invested on weekly bars, which explains their low returns. Many results rest on very few trades, so this table alone does not prove that one strategy is better.

5 Backtesting engine

Execution. A signal is computed at the close of a bar and executed at the **Open of the next bar**, so it cannot use a price that did not exist yet. Positions are long only and use all the cash. With a slippage s and a fee f, a buy is filled at $p(1+s)$, with quantity $q = \text{cash}/(\text{fill}(1+f))$, and a sale at $p(1-s)$, with proceeds $q\,\text{fill}\,(1-f)$. Bars before the requested period only warm up the indicators.

Protective exits. The stop level is the highest of $E(1-\text{SL})$ and $\text{peak}(1-\text{TS})$, and the target is $E(1+\text{TP})$, with E the entry price. If a bar opens beyond a level, the exit is at the Open. If stop and target are both touched in one bar, the order is unknown, so we prudently assume the stop came first.

Metrics. We compute 13 metrics for the strategy and for buy & hold, including the cumulative and annualized return, volatility, maximum drawdown, win rate, profit factor and exposure. With P periods per year (252 or 52) and r_t the returns:

$$\text{Sharpe} = \frac{\bar{r} - r_f}{\sigma(r)}\sqrt{P}, \qquad \text{Sortino} = \frac{\bar{r} - r_f}{\sqrt{\frac{1}{n}\sum \min(0, r_t - r_f)^2}}\sqrt{P}, \qquad \text{Calmar} = \frac{\text{annualized return}}{\text{max drawdown}}.$$

Heatmap. Up to 625 pairs of parameters go through the same validation and engine as a normal backtest. An optimiser always finds a best cell, even on noise, so a warning appears when the best value is more than twice the average of its neighbours.

Checking the numbers. Expected values in the 184 engine tests are recomputed from the formulas, never copied from the output. Scenarios were also computed independently in a spreadsheet (Figure 3). Property tests on random signals check that the equity is never negative, that the final equity equals the capital plus the profits, and that truncating the future does not change the past.

Scenario S1 - signals B H S H H H - hand check (capital 10 000, fee 0.1 %, no slippage)

To compare with the engine tests: quantity 97.94, net profit 371.39, fees 20.37, equity 10087.95 / 10283.83 / 10371.39

Capital	10000,		input
Fee rate	0,001		input (0.1 %)
Entry price (Open of bar 1)	102,		input: Buy signal on bar 0, executed at the next Open
Exit price (Open of bar 3)	106,		input: Sell signal on bar 2, executed at the next Open
Quantity	97,9413		capital / (entry x (1 + fee)) [rule X6]
Cost of the purchase	10000,0000		q x entry x (1 + fee)
Proceeds of the sale	10371,3933		q x exit x (1 - fee) [rule X6]
Fees (entry + exit)	20,3718		q x entry x fee + q x exit x fee
Net profit	371,3933		proceeds - cost [rule X13]
Return of the trade	0,0371		net profit / cost
Bar	**Close**	**Equity**	**How it is computed**
Bar 0	101	10000,0000	first point = capital [rule X11]
Bar 1	103	10087,9513	bought at the Open of bar 1, valued at the Close: q x Close
Bar 2	105	10283,8338	q x Close
Bar 3	104	10371,3933	sold at the Open of bar 3: everything is cash
Bar 4	108	10371,3933	cash
Bar 5	110	10371,3933	cash

Figure 3: A six-bar scenario recomputed by hand: quantity, fees, net profit and equity.

6 Robustness analysis

A good backtest proves little on its own, so we added two tools.

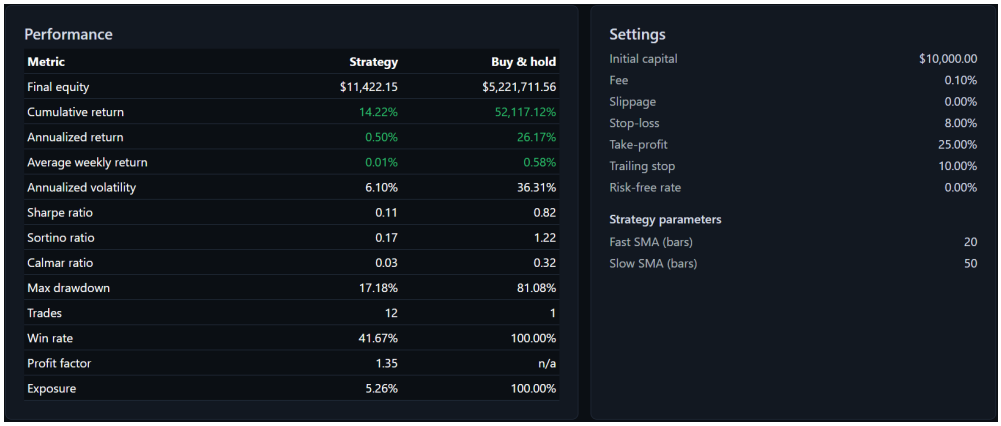
Train/test. The page optimises on the first part of the period (70 % by default) and tests the best parameters once on the rest. On AAPL weekly since 2010, the crossover with fast 26 and slow 35 has a Sharpe ratio of 1.02 in training and 0.75 in test. This drop is expected, since the parameters were chosen on the training data. With 5 and 6 trades, neither figure is statistically solid.

Monte Carlo. We draw the returns of the trades at random with replacement and compound them, 100 to 10,000 times, from a fixed seed. This gives distributions of the final return and of the trade-level drawdown. *Consecutive down days* on AAPL weekly has a 68.7 % probability of loss, and its actual result (-41.98 %) is near the median: it is simply a losing strategy. The crossover

on MSFT weekly has an 8.5 % probability of loss, but percentiles ranging from -17.4 % (P5) to +29,111 % (P95): with 15 trades, one of which earned +521 %, the result depends heavily on that single trade.

7 Web application

The site has six pages: Backtest, Compare, Optimize, Validate, Strategies and Data. The video shows them in use. The web project contains no financial logic: controllers call the services and view models format the values. Forms and strategy cards are generated from the catalog, so no strategy is hard-coded. Only the exceptions of the contract are shown to the user, with the input kept; anything else is a bug and reaches a generic error page. Every result has a `GET` URL holding all its settings, and one class writes a single canonical form of it, so a result can be shared and reproduced. Bootstrap and Chart.js are served locally.



Performance

Metric	Strategy	Buy & hold
Final equity	$11,422.15	$5,221,711.56
Cumulative return	14.22%	52,117.12%
Annualized return	0.50%	26.17%
Average weekly return	0.01%	0.58%
Annualized volatility	6.10%	36.31%
Sharpe ratio	0.11	0.82
Sortino ratio	0.17	1.22
Calmar ratio	0.03	0.32
Max drawdown	17.18%	81.08%
Trades	12	1
Win rate	41.67%	100.00%
Profit factor	1.35	n/a
Exposure	5.26%	100.00%

Settings

Initial capital	$10,000.00
Fee	0.10%
Slippage	0.00%
Stop-loss	8.00%
Take-profit	25.00%
Trailing stop	10.00%
Risk-free rate	0.00%

Strategy parameters

Fast SMA (bars)	20
Slow SMA (bars)	50

Figure 4: Results of a real backtest: AAPL weekly, moving average crossover with stops.

8 Reports and deployment

The PDF report is built with QuestPDF and ScottPlot (Figure 5). Every number comes from `BacktestResult` and nothing is recomputed, so the report cannot disagree with the screen. We checked one real report by hand: for MSFT weekly, the 15 net profits add up to 85,964.29, which matches the printed final equity of 95,964.29. Trades can also be exported as CSV.

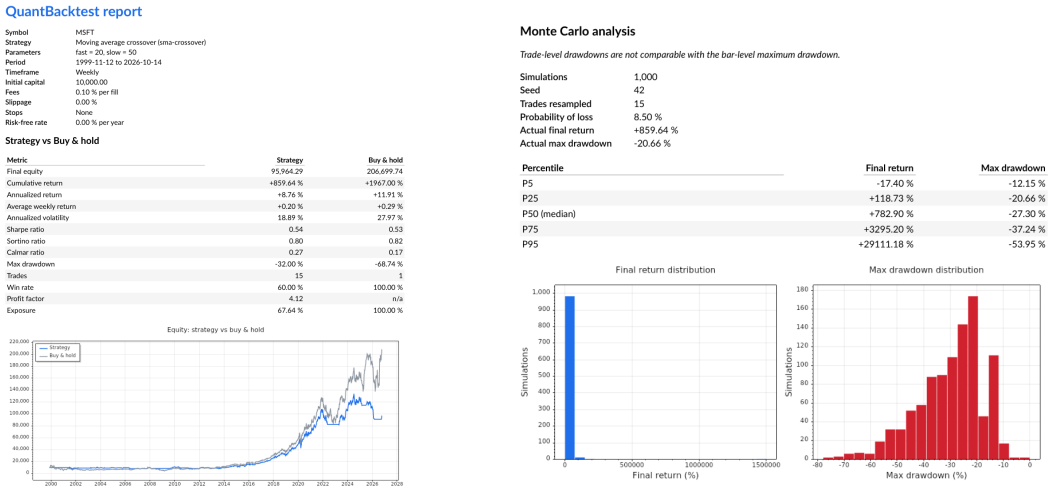


Figure 5: PDF report: summary page and Monte Carlo page.

`docker compose up -build` starts the application on port 8080. The image is only built if the solution compiles with warnings as errors and all the tests pass. It runs as a non-root user, keeps the database in a named volume and reads the API key from a local `.env` file that is never committed. We ran it with Docker Desktop, including a real API refresh and the generation of a PDF.

9 Quality and teamwork

The solution has 1,494 automated tests with no network and no real clock: Web 923, Engine 184, Strategies 168, Data 124, Reports 55, Analytics 14, Fakes 14, Core 12. Each test quotes the rule it checks. Razor views, JavaScript and CSS were checked by hand.

All changes went through pull requests, on branches named after each part: 42 were merged between 13 October and 8 November 2026. GitHub Actions compiles with warnings as errors, runs all tests, checks the title of the pull request and verifies that the changed files belong to the part of the branch. Branch protection requires these checks and forbids force-pushes. We removed the mandatory approval on 15 October because it slowed us down, while keeping it for contract changes; the CI cannot replace a human reading the code, so this was a compromise.

Our most useful decision was to write a precise contract before the code. We also learned that a backtest can be wrong in silent ways (a look-ahead, a misread decimal, an error saved as data), which is why the tools showing the limits of a result matter as much as the strategies.

10 Limitations and future work

- **Data.** Only 100 raw daily bars are free, so long tests are weekly; seed files are snapshots of October 2026. A paid source would give long adjusted daily histories.
- **Simulation.** Long only, no leverage, no volume limit, no taxes and no interest on cash. Inside a bar, the order of the High and the Low is unknown.
- **Validation.** Train/test uses a single split, and walk-forward validation would be better. Monte Carlo resamples one backtest and assumes independent trades.
- **Interface.** Only the equity chart has a log scale, and the comparator uses default parameters.

- **Method.** Our symbols are large, successful companies chosen with hindsight, on which buy & hold is hard to beat. Past results do not predict future results.

A User manual

Start. With Docker: copy `.env.example` to `.env`, optionally add a free Alpha Vantage key, run `docker compose up -build` and open `http://localhost:8080`. Without Docker: set the key with `dotnet user-secrets set "AlphaVantage:ApiKey" "<key>" -project src/QuantBacktest.Web`, then run `dotnet run -project src/QuantBacktest.Web` and open `http://localhost:5080`. Without a key, AAPL, MSFT, SPY and JPM still work.

Page	Use
Backtest	Choose a symbol, timeframe, strategy and settings. Empty stops are disabled. The result offers the PDF, the CSV, Monte Carlo and optimisation.
Compare	Run 2 to 5 strategies with default parameters on the same data.
Optimize	Test pairs of two parameters. An "isolated peak" warning suggests overfitting.
Validate	Optimise on the first part of the period and test on the rest.
Strategies	Read each strategy's rules and risks, and create no-code strategies.
Data	See cached series and their quality report, and refresh a series (1 of 25 daily calls).

C# strategy. Copy `Custom/TemplateStrategy.cs` into the `Strategies` folder, remove `[HiddenStrategy]`, set a unique id, fill in the rules and rebuild.

Common messages. *API key not configured*: set the key, or use a symbol with a seed file. *Request limit reached*: wait for midnight UTC or use cached symbols. *Prices are not adjusted*: normal for daily data; use weekly for long tests. *Not enough trades for Monte Carlo*: at least 5 trades are needed.